

RAG 3일 특강

Day 1 — RAG 기초와 첫 구현

오늘의 로드맵

- M1.1 LLM의 한계와 RAG의 필요성
- M1.2 RAG 아키텍처 개요
- M1.3 임베딩과 벡터 검색 기초 + Lab1.1
- M1.4 문서 로딩과 청킹 전략 + Lab1.2
- M1.5 검색과 생성 연결 + Lab1.3

LLM은 어떻게 답을 만드는가

- 다음에 올 토큰을 하나씩 예측해서 문장을 완성 — 이걸 "자기회귀적(autoregressive) 생성"이라고 부름
- 예: "오늘 날씨가" 다음에 "좋다"가 올 확률이 높다고 학습된 패턴대로 이어붙이는 것
- 지식은 학습 시점에 파라미터(숫자 수십억~수천억 개) 안에 "고정"되어 저장됨 — 학습이 끝나면 그 이후 세상 일은 절대 모름
- 결정적으로, 모델 구조 어디에도 "모른다"에 해당하는 토큰/메커니즘이 없음 — 확률이 낮아도 그중 가장 높은 걸 계속 골라서 이어붙일 뿐
- 이 두 가지가 겹치면? → 모르는 내용도 "그렇듯하게 끝까지 이어붙이는" 결과가 나옴 (바로 다음 슬라이드의 사고 원인)

실제 할루시네이션 사례

- 사건: *Mata v. Avianca, Inc.* (미국 뉴욕 남부지방법원, 2023) — 변호사 Steven Schwartz가 챗GPT가 지어낸 판례 6건을 그대로 법원에 제출, \$5,000 제재
- 출처: [Mata v. Avianca, Inc. — Wikipedia](#) / [CBS News 보도](#)
- 존재하지 않는 라이브러리 API를 그럴듯하게 지어내는 코드 생성 (개발자에게 더 와닿는 사례)
- 공통점: 모델이 "모른다"고 말하지 않고 매번 그럴듯한 답을 완성해서 내놓음

라이브 재현: 없는 API 지어내기

- 지금 볼 것: 세상에 없는 API 이름을 마치 실제로 있는 것처럼 물어보면, 모델이 "모른다"는 대신 어떻게 반응하는지 실시간 확인
- 시연 질문: "Spring AI의 VectorStore에 있는 deleteAllByMetadata() 메서드 사용법을 알려줘" (이 메서드는 실존하지 않음)
- 관찰 포인트: 존재하지 않는 메서드인데 모델이 코드를 만들어냄 ("모른다"고 하지 않음)
- 실측: 실제로 `VectorStore vectorStore = new VectorStore();`처럼 생성자까지 지어내고, `deleteAllByMetadata("metadataId", Map.of(...))` 같은 가짜 시그니처를 자신 있게 제시함

라이브 검증: 실제 인터페이스 확인하기

- 목적: 방금 모델이 지어낸 메서드가 정말 존재하지 않는지, 실제 라이브러리 클래스 파일로 직접 검증
- 방법: 우리 프로젝트가 실제로 받은 spring-ai-vector-store-2.0.0.jar에서 VectorStore.class를 꺼내 javap로 디컴파일
- 결과: deleteAllByMetadata()는 어디에도 없음 — 모델이 완전히 지어낸 이름
- 흥미로운 지점: delete(Filter.Expression)이라는, 메타데이터 조건으로 삭제하는 진짜 메서드는 있음 — 모델이 "그럴듯한 기능이 있을 법한 이름"을 절반쯤 맞는 감으로 지어낸 것에 가까움 (완전 랜덤이 아니라 더 위험함)

```
unzip -o spring-ai-vector-store-2.0.0.jar \
org/springframework/ai/vectorstore/VectorStore.class
javap org/springframework/ai/vectorstore/VectorStore.class
```

```
public interface org.springframework.ai.vectorstore.VectorStore ... {
    public abstract void add(java.util.List<Document>);
    public abstract void delete(java.util.List<String>);
    public abstract void delete(Filter.Expression);
    public default void delete(String);
    public default <T> Optional<T> getNativeClient();
    ...
}
```

라이브 재현: 최신 이벤트 질문

- 시연 질문: "어제 발표된 최신 Spring Boot 패치 버전이 뭐야?"
- 관찰 포인트: 실제 버전을 그럴듯하게 지어내고, 날짜/기능까지 구체적으로 상상해서 답함
- 실측: "Spring Boot 3.1.11이 2023년 12월에 발표됐다"는 식으로 실재하지 않는 버전/날짜를 정색하고 제시, 심지어 존재하지 않는 세부 기능 목록까지 만들어냄

Private/사내 데이터 문제

- 학칙, 사내 위키처럼 애초에 학습 데이터에 없는 지식
- 보조 예시: 시연 질문 "우리 학교 2학기 수강신청 정정기간이 며칠이야?" → 지어낸 날짜로 답함
- 인터넷에 공개되지 않은 정보는 모델이 절대 알 수 없다는 것이 핵심

대안 ① Fine-tuning의 한계

Fine-tuning

비용 큼, 데이터 준비 부담, 모델 재학습마다 최신성 다시 깨짐

VS

RAG

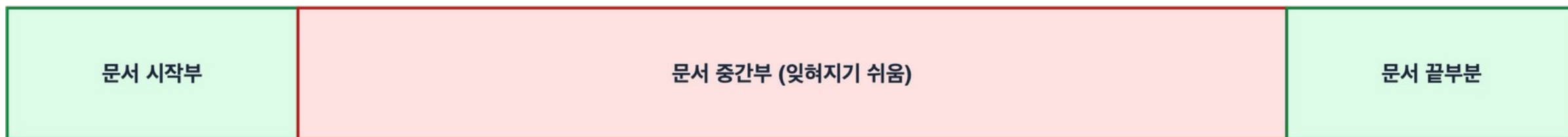
모델은 그대로, 문서만 갱신하면 최신성 유지

- Fine-tuning은 "GPU 학습 비용 + 라벨링된 데이터셋"이 필요, RAG는 문서 폴더에 파일 하나 추가하면 끝

대안 ② 문서 통째로 프롬프트에 넣기의 한계

- 문제: 비용 폭증(토큰 수 비례), "lost-in-the-middle" — 중간에 낀 정보를 모델이 무시
- 문서 전체를 프롬프트에 붙이기: 간단하지만 비용/컨텍스트 한계
- RAG: 필요한 부분만 골라서 넣기 때문에 비용/정확도 둘 다 개선

긴 컨텍스트를 통째로 프롬프트에 넣었을 때 LLM의 주의(attention) 분포



정답이 중간부에 있으면, 앞/뒤에 있을 때보다 LLM이 놓치기 쉬움 (lost-in-the-middle 현상)

정확도: 높음 —————▶ 낮음 ◀————— 높음

lost-in-the-middle은 왜 생기는가

- 원인 ①: 학습 데이터의 구조적 편향 — 사람이 쓴 글은 핵심 정보를 보통 서두(도입/주제문)나 결론에 배치함 → 모델이 "중요한 건 앞/뒤에 있다"는 패턴을 통계적으로 학습
- 원인 ②: "attention sink" 현상 — 문서 맨 앞쪽 토큰들이 의미와 무관하게 유독 많은 attention을 받는 경향이 실제로 관찰됨 (Xiao et al., "Efficient Streaming Language Models with Attention Sinks", 2023)
- 원인 ③: attention 희석 — softmax 특성상 컨텍스트가 길어질수록(=경쟁하는 토큰이 많아질수록) 토큰 하나하나가 받는 attention 가중치의 절대량이 작아짐 → 중간 토큰이 상대적으로 더 손해
- 정직한 한계: 이걸 여전히 활발히 연구 중인 주제이고, 아래 원인들이 "완전히 증명된 유일한 이유"는 아님 — 다만 여러 연구에서 일관되게 관찰되는 현상이라는 점은 확실함
- 원 논문: Liu et al., "Lost in the Middle: How Language Models Use Long Contexts" (2023)

RAG란 무엇인가

- 핵심 정의: "모델은 그대로 두고, 답변 근거를 실시간으로 찾아 프롬프트에 넣어준다"
- Retrieval(검색) + Augmented(증강) + Generation(생성) 세 단어의 합성어
- 이미 있는 검색 기술 + 이미 있는 LLM을 조합한 것 — 새로운 발명이 아니라 조합

그런데 왜 이게 실제로 통하는가

- 의문: 모델은 여전히 "다음 토큰 예측"만 함(맨 처음에 배운 것) — 그런데 왜 문서 몇 줄 끼워 넣었다고 갑자기 정확해지나?
- 비유: 클로즈북 시험 → 오픈북 시험으로 바뀌주는 것
- RAG 없이 답하기 = 암기한 것만으로 시험 보기 (파라미터 속 흐릿한 통계적 기억에서 답을 "복원"해야 함 → 잘못 복원하면 할루시네이션)
- RAG로 답하기 = 답이 적힌 페이지를 펴놓고 시험 보기 (정답 텍스트가 지금 입력에 실제로 존재 → "기억해내기"가 아니라 "베껴서 정리하기"만 하면 됨, 훨씬 쉽고 안정적인 작업)
 - 트랜스포머의 attention이 정확히 이 역할 — 생성할 때 입력에 있는 관련 토큰(=검색된 문서)에 직접 "집중"해서 그 내용에 근거해 다음 토큰을 예측함
 - 중요한 한계: 이건 "모른다"를 가능하게 만드는 게 아니라, "정답이 입력 안에 있을 때 더 잘 베끼게" 해주는 것뿐 — 검색된 문서에도 답이 없으면 모델은 여전히 지어냄 (Lab1.3에서 직접 목격하고, M2.6에서 프롬프트로 보완)

RAG가 해결하는 4가지

최신성	문서만 갱신하면 항상 최신 정보로 답변
사실 기반	실제 문서에서 검색된 내용을 근거로 답하니 출처 제시 가능
도메인 특화	사내 문서, 학칙처럼 공개 안 된 지식도 답변 가능
비용	파인튜닝 대비 훨씬 저렴하고 빠르게 적용 가능

미니 데모: 순수 LLM vs RAG 챗봇

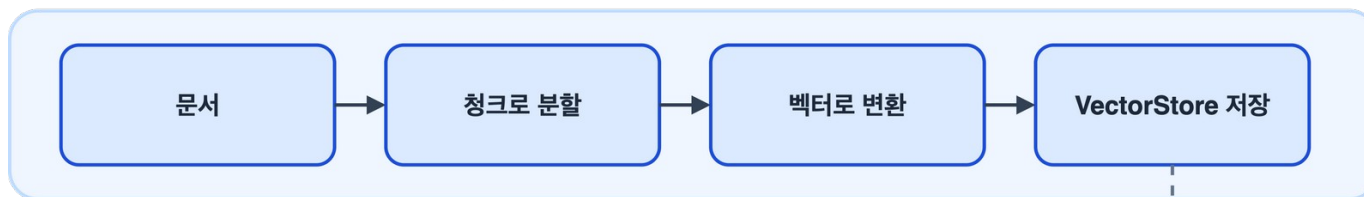
- 지금 볼 것: 같은 질문을 (a) 순수 LLM (b) RAG 챗봇에 각각 던져서 답변을 나란히 비교
- 시연 질문: "MCM-200 커피메이커의 E2 오류 코드가 뜨면 어떻게 해야 해?" (세상에 없는 가상 제품이라 모델이 학습했을 리 없음)
- 실측: (a) 순수 LLM은 "10분 대기 후 재시작" 등 완전히 지어낸 절차를 자신 있게 제시 / (b) RAG는 문서 제5조 근거로 "온도 센서 이상, 고객센터(1544-0000) 문의"로 정확히 답함
- 관찰 포인트: 두 답변이 pureAnswer/ragAnswer로 한 화면에 나란히 나옴 — 근거가 있는 답과 없는 답의 차이가 바로 비교됨

M1.2 RAG 아키텍처 개요

전체 그림: 두 개의 파이프라인

- 인덱싱 파이프라인 (오프라인, 미리 준비) vs 질의응답 파이프라인 (온라인, 질문마다 실행)
- 인덱싱은 "책을 도서관에 등록하는 과정", 질의응답은 "독자가 책을 찾아 읽는 과정"에 비유 가능
- 두 파이프라인은 완전히 다른 시점에 실행됨 — 인덱싱은 서비스 배포 전/문서 갱신 시 1회, 질의응답은 사용자 질문마다 매번
- 실습에서: 오늘 Lab1.2가 인덱싱 파이프라인을, Lab1.3이 질의응답 파이프라인을 각각 코드로 구현
- 흔한 착각: "질문할 때마다 문서를 다시 읽고 임베딩한다"고 생각하는 것 — 실제로는 인덱싱은 미리 끝나 있고, 질문 때는 이미 저장된 벡터를 검색만 함

인덱싱 파이프라인 (오프라인 · 사전 준비)



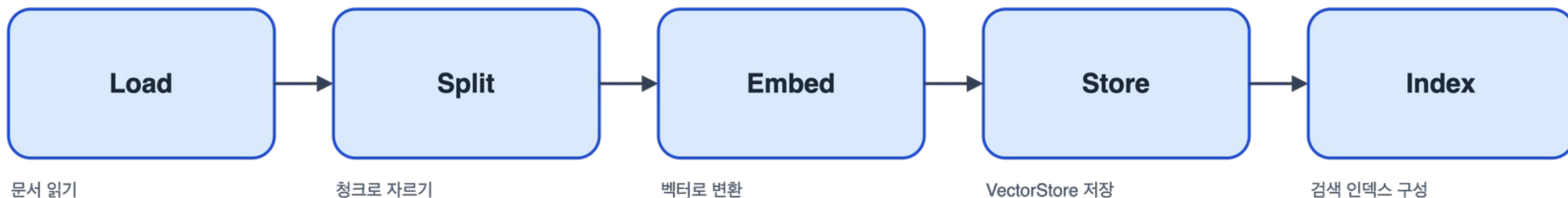
질의응답 파이프라인 (온라인 · 질문마다 실행)



나중에 검색할 때 사용

인덱싱 파이프라인 5단계

- Load → Split → Embed → Store → Index
- Load: 문서를 읽어온다 / Split: 청크로 자른다 / Embed: 벡터로 바꾼다 / Store: 저장한다 / Index: 검색 가능하게 색인한다
 - 이 5단계는 문서가 바뀔 때만 다시 실행하면 됨 (질문마다 반복 X)
- 5단계 중 제일 오래 걸리는 건 보통 Embed — 청크 수만큼 임베딩 모델 API를 호출해야 하기 때문 (문서가 클수록 체감됨)
- 실무에서는 이 전체를 배치 작업으로 돌림 — 매번 서버가 켜질 때마다 재실행하지 않음



Split: 청크 크기를 어떻게 정하나

- 청크가 너무 작으면 — 문장이 앞뒤 문맥과 분리되어 검색은 되지만 의미가 불완전해짐 (예: "이 경우 30일 이내 환불"만 뽑히고 "이 경우"가 뭉치는 다른 청크에 있음)
- 청크가 너무 크면 — 한 청크에 여러 주제가 섞여 임베딩이 흐려지고, 관련 없는 내용까지 딸려와 컨텍스트를 낭비 (앞서 본 lost-in-the-middle 현상과 직결)
- 오버랩(overlap) — 청크 경계에서 문장이 잘리지 않도록 앞 청크 끝부분을 다음 청크 앞에 10~20% 겹쳐서 자르는 게 일반적
- 기준 단위는 글자 수가 아니라 토큰 수 — 모델마다 토큰라이저가 달라 같은 글자 수도 토큰 수가 다름
- 정답은 없음 — 문서 종류(FAQ vs 논문 vs 코드)마다 적정 크기가 다르고, 실제 검색 품질을 측정해가며 조정 (Day2에서 직접 튜닝)

Embed: 모델마다 뭐가 다른가

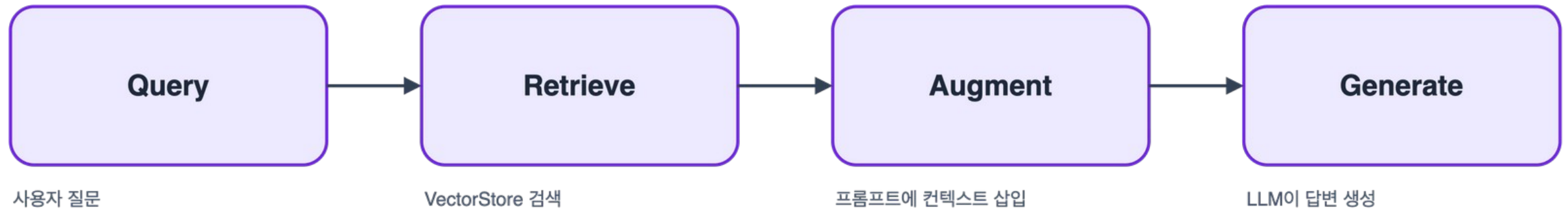
bge-m3 (오늘 실습 사용)	1024차원 · 다국어 100개 이상 언어 지원 · 최대 8192 토큰까지 한 번에 임베딩
OpenAI text-embedding-3-small	1536차원 · 영어 중심 학습 · dimensions 파라미터로 256~1536까지 축소 가능
OpenAI text-embedding-3-large	3072차원 · small보다 정교한 의미 구분, 대신 저장 공간·연산 비용 증가
선택 기준	언어 지원 범위, MTEB 벤치마크 점수, 최대 입력 길이, API 비용/지연시간 네 가지를 함께 고려

실제 예시: 우리 실습 문서가 이렇게 쪼개진다

- 오늘 실습 문서(MCM-200 설명서)를 chunk size 300 토큰으로 자르면 실제로 이렇게 4개로 나뉨 (Lab1.2에서 직접 재현)
- 청크 0 끝: "...3. 전원 버튼을 눌러 추출을 시작합니다." → 청크 1 시작: "추출은 약 6~8분 소요됩니다. 제3조 (보증 및 환불 정책)..."
- 문장 자체는 안 잘렸지만 "추출 방법"과 "소요 시간"이라는 한 덩어리 정보가 서로 다른 청크로 갈라짐 — 검색 시 한쪽만 뽑혀 나올 수 있음
- 토큰화 후 다시 텍스트로 복원하는 과정에서 글자가 미세하게 유실되기도 함 — 실제 결과: "환불은... 가능하며, 제품 박스와 구성품이 모두 보존"이 "환불은... 가능 모두 보존"으로 줄어듦
- 이건 지어낸 예시가 아니라 Lab1.2에서 여러분이 직접 콘솔에 출력해서 그대로 재현하게 될 실제 실행 결과

질의응답 파이프라인 4단계

- Query → Retrieve → Augment → Generate
- Query: 사용자 질문 / Retrieve: 저장된 벡터에서 유사한 것 검색 / Augment: 검색 결과를 프롬프트에 삽입 / Generate: LLM이 최종 답변 생성
- 이 4단계는 질문이 들어올 때마다 매번 처음부터 다시 실행됨

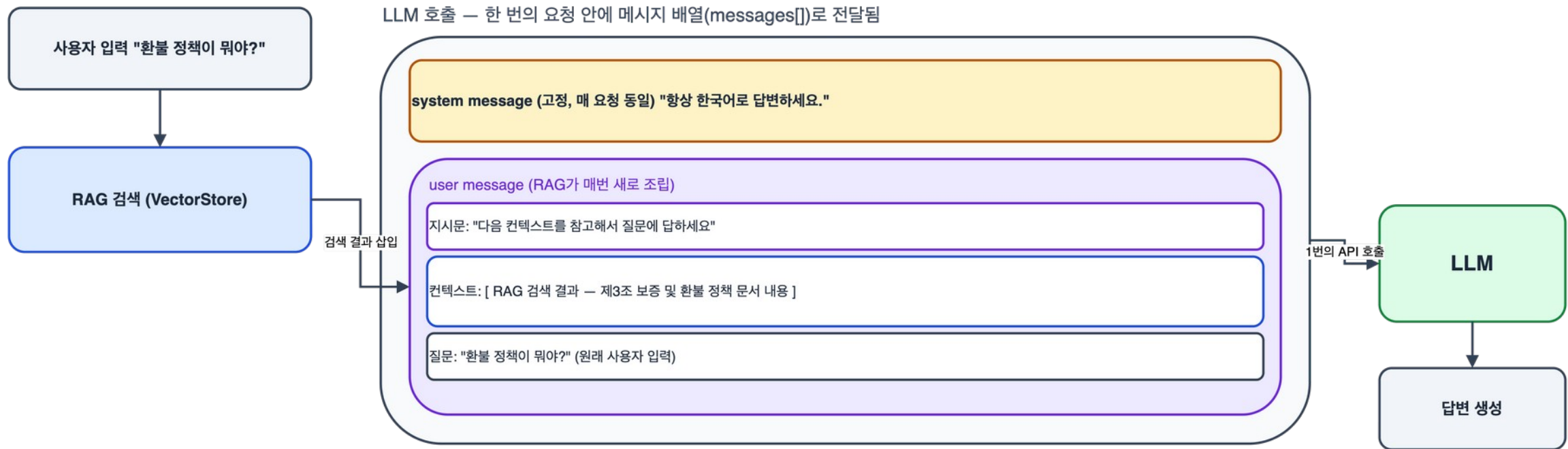


Spring AI 컴포넌트 지도

DocumentReader	Load 단계 담당
TextSplitter	Split 단계 담당
EmbeddingModel	Embed 단계 담당
VectorStore	Store + Index + Retrieve 단계 담당 (양쪽 파이프라인에 걸침)
ChatClient + Advisor	Augment + Generate 단계 담당

"증강(Augmented)"의 실제 의미 — 메시지 구조

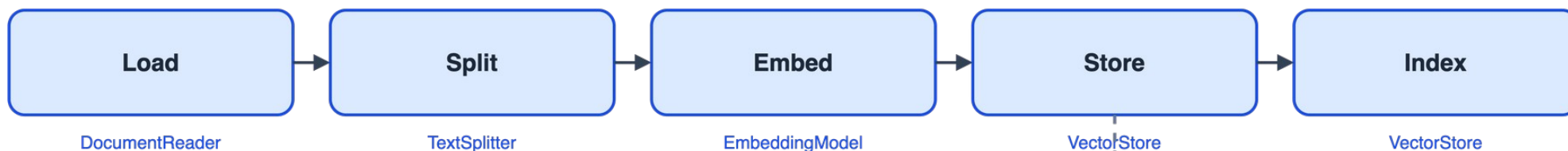
- LLM 호출은 한 번의 요청 안에 system message + user message로 구성된 배열(messages[])을 통째로 보냄
- system message: 매 요청 고정 (예: "항상 한국어로 답변하세요")
- user message: RAG가 매번 새로 조립 — 지시문 + 검색된 컨텍스트 + 원래 질문이 하나로 합쳐짐
- 사용자 눈에는 원래 짧은 질문만 보이지만, LLM이 실제로 받는 건 이렇게 조립된 훨씬 긴 메시지



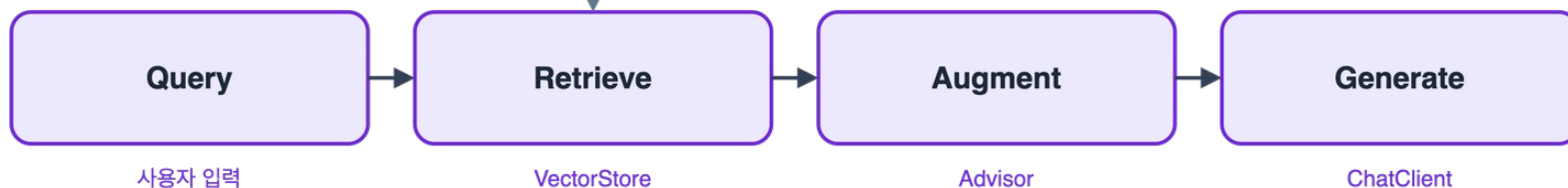
종합: 9단계 파이프라인 전체 지도

- 지금까지 따로 본 두 파이프라인과 Spring AI 클래스 매핑을 한 장으로 합침
- 인덱싱 5단계(Load~Index)와 질의응답 4단계(Query~Generate)가 VectorStore로 연결되는 게 한눈에 보임
- 이 한 장이 오늘 하루 실습(Lab1.1~1.3)에서 실제로 만들 것의 전체 지도
- 강사 진행 팁: 학생들에게 "다음 단계가 뭘까요?"를 하나씩 물어보며 빈 화면에서 이 그림을 함께 그려나가는 방식으로 진행 가능 (정답은 이 슬라이드)

인덱싱 파이프라인 (오프라인 · 문서가 바뀔 때만 실행)



질의응답 파이프라인 (온라인 · 질문마다 매번 실행)



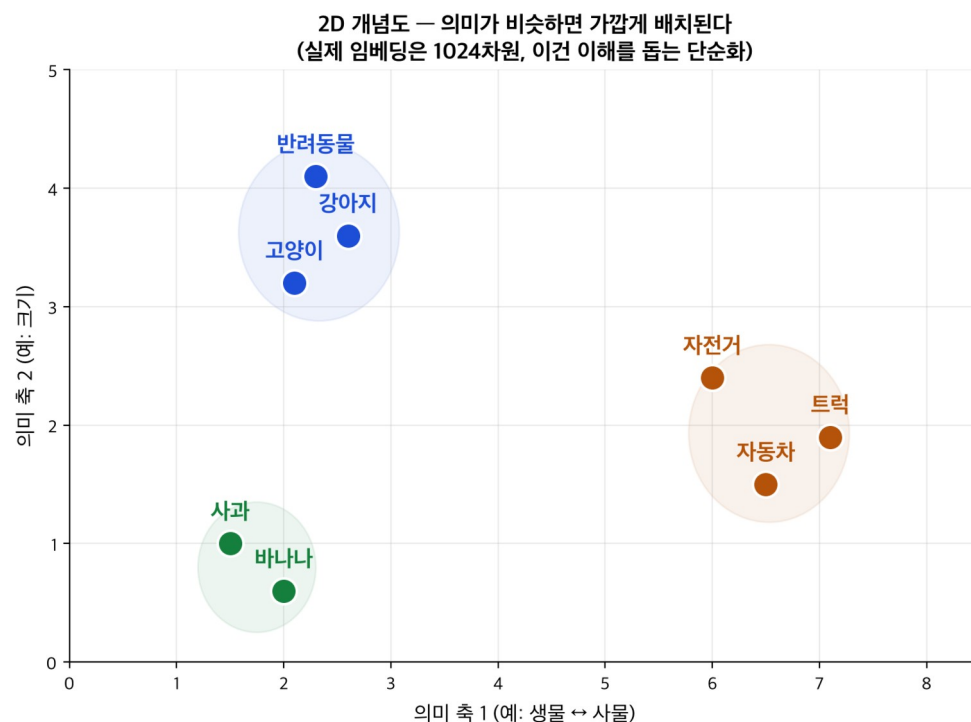
3일 로드맵 + 자주 헛갈리는 것

- Day1 기본 구현 → Day2 검색 품질 → Day3 고급 패턴/평가
 - 주의: RAG ≠ 검색결과를 프롬프트에 단순 붙여넣기 (Advisor 패턴이 다음에 나옴)
- Day1의 목표는 "일단 동작하는 것", Day2의 목표는 "정확하게 동작하는 것"

M1.3 임베딩과 벡터 검색 기초

벡터란 무엇인가

- 숫자 배열로 의미를 표현한다는 것
- 보조 이미지: 2D 좌표평면에 단어들을 점으로 배치한 비유 그림
- 예: "강아지"와 "고양이"는 좌표평면에서 가깝게, "강아지"와 "자동차"는 멀게 배치된다고 상상



임베딩 모델의 역할

- 텍스트 → 고정 길이 숫자 벡터로 변환
- 차원 수 개념 (384 / 768 / 1024) — 숫자가 몇 개 들어있는 배열인지
- 문장이 길든 짧은 임베딩 모델에 넣으면 항상 같은 길이의 벡터가 나옴

의미적 유사도 vs 키워드 일치

키워드 검색

"자동차"와 "차량"을 다른 단어로 취급

vs

임베딩 검색

"자동차"와 "차량"을 의미적으로 가깝게 인식

- 전통적인 Ctrl+F나 SQL LIKE 검색은 정확히 같은 글자가 있어야만 찾아냄

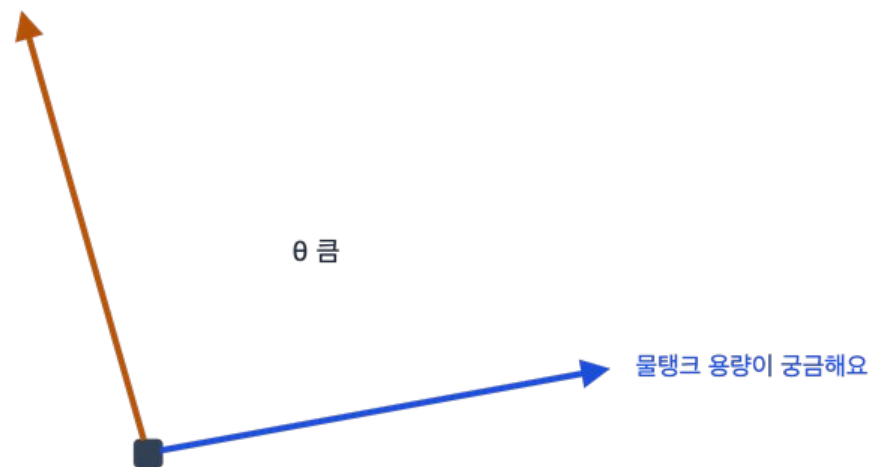
코사인 유사도

- 두 벡터 사이의 "각도"로 유사성을 잴다는 직관
- 보조 이미지: 각도 그림 (0도=완전히 같음, 90도=무관)
- 값의 범위는 보통 -1~1, 1에 가까울수록 의미가 비슷함

유사한 문장 — 각도(θ) 작음 $\rightarrow$ 코사인 유사도 높음



무관한 문장 — 각도(θ) 큼 $\rightarrow$ 코사인 유사도 낮음



유클리드 거리 vs 코사인 유사도

- 임베딩에서는 벡터의 "방향"이 중요 → 코사인 유사도를 쓰는 이유
- 유클리드 거리는 벡터의 "크기"에도 영향을 받아서 문장 길이가 다르면 왜곡될 수 있음
- 대부분의 VectorStore가 기본값으로 코사인 유사도를 채택하는 이유이기도 함

Spring AI `EmbeddingModel` 인터페이스

- EmbeddingResponse 구조 간단 소개 — 메타데이터(모델명, 토큰 사용량 등)까지 포함하는 확장 버전
- Spring AI에서는 OpenAI든 Ollama든 이 인터페이스 하나로 통일해서 다룸

```
public interface EmbeddingModel {  
    float[] embed(String text);  
    // 더 상세한 정보가 필요하면:  
    // EmbeddingResponse embedForResponse(List<String> texts);  
}
```

Ollama 임베딩 모델 & 한국어 이슈

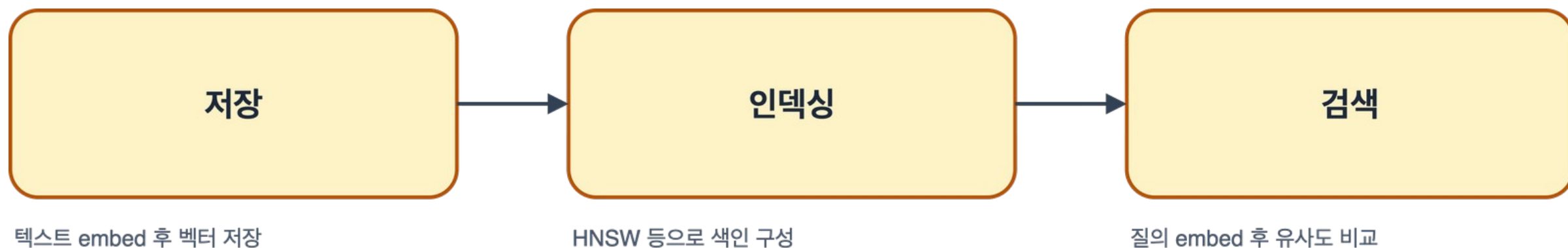
- 소개: nomic-embed-text, bge-m3
- 실측 경고: nomic-embed-text는 한국어 의미 구분에 실패(무관한 문장이 더 유사하게 나옴) → 이 강의는 bge-m3로 통일해서 진행
- 실측 수치: "물통 용량" 유사 문장 0.67 vs "저녁 메뉴" 무관 문장 0.78(더 높음, 잘못됨) → bge-m3는 0.62 vs 0.44로 정상

벡터 검색 개념

- 최근접 이웃 탐색(Nearest Neighbor)
- ANN(HNSW) — 정확도를 살짝 포기하고 속도를 얻는 근사 탐색
- 문서가 수백만 개면 완전탐색은 너무 느려서 근사 알고리즘이 필수

VectorStore 내부 동작 미리보기

- 저장(embed+저장) + 인덱싱(HNSW 등) + 검색(질의 embed 후 유사도 비교) 3단계
- VectorStore는 이 3단계를 개발자가 신경 쓰지 않아도 되게 캡슐화해줌



Top-K 검색

- "가장 가까운 K개만 가져온다"는 개념과 실제 활용 예
- K를 너무 작게 잡으면 정답이 누락되고, 너무 크게 잡으면 무관한 내용까지 프롬프트에 섞임
- 오늘 실습에서는 기본값으로 K=5 정도를 사용

정리 + Lab1.1 예고

- 벡터 = 의미를 숫자로 표현한 것 / 코사인 유사도 = 두 벡터의 유사성을 재는 방법
- VectorStore = 저장+인덱싱+검색을 캡슐화한 컴포넌트
- 다음: 이 모든 걸 직접 코드로 확인하는 첫 실습

Lab 1.1 — 프로젝트 셋업 + 임베딩 유사도 실습

- 목표: Spring AI + Ollama 연결하고 실제로 임베딩을 호출해본다
- 체크리스트: 프로젝트 생성 → yml 설정 → Ollama 모델 확인 → 코드 작성 → 실행
- 준비물: JDK 17+, Ollama 설치 및 bge-m3 모델 다운로드 완료 상태

의존성 & application.yml

- base-url은 로컬 Ollama 서버 주소 — 기본값 그대로 사용
- embedding.options.model과 chat.options.model을 분리해서 지정 가능 (임베딩용/생성용 모델이 다를 수 있음)

```
org.springframework.ai:spring-ai-starter-model-ollama
```

```
spring:
  ai:
    ollama:
      base-url: http://localhost:11434
      embedding.options.model: bge-m3
      chat.options.model: llama3.2:3b
```


EmbeddingModel 호출 코드

- EmbeddingDemoService — embeddingModel.embed(text) 호출
- embed()는 float[]를 반환 — 이 배열 하나가 문장 하나의 "의미 좌표"
- 코드 출처: rag-day1-demo/.../lab11/EmbeddingSimilarityDemo.java

실행 화면: Spring이 뜨는 모습을 직접 본다

- 실행 명령: `./mvnw spring-boot:run -Dspring-boot.run.profiles=lab11`
- Spring Boot 특유의 배너와 함께 애플리케이션이 부팅됨
- 0.5초 만에 뜸 — 무거운 프레임워크가 아니라는 걸 체감하는 포인트
- 어떤 프로파일(lab11)이 활성화됐는지 로그로 바로 확인 가능

```
  /\ \ / ____' _ _ _ _ _ ( _ _ _ _ _ \ \ \ \ \
 ( ( ) \ ____| ' _ | ' _ | | ' _ \ \ \ \ \ \
  \ \ / ____| |_) | | | | | | | | | | ( _ | | ) ) )
  ' | ____| . _ | | | | | | | | | | \ \ \ \ \ /
  =====|_|=====|____/=/=/=/=/
```

```
:: Spring Boot ::                (v4.1.0)
```

```
INFO --- RagDay1DemoApplication : Starting RagDay1DemoApplication using Java 23
INFO --- RagDay1DemoApplication : The following 1 profile is active: "lab11"
INFO --- RagDay1DemoApplication : Started RagDay1DemoApplication in 0.517 seconds
```

임베딩 벡터 차원 확인 (실행 결과)

- 실행 결과: 벡터 차원 수 1024, 앞부분 값 [-0.0034, -0.0149, -0.0400, ...]
- 관찰 포인트: 이 숫자(1024)를 Lab2.1 PGVector 설정에서 그대로 써야 함
- 값 하나 하나는 사람이 보서는 의미를 알 수 없는 숫자지만, 모델 입장에서는 의미가 응축된 좌표

코사인 유사도 함수 구현

- `cosineSimilarity(float[] a, float[] b)` — dot product / (norm × norm)
- 세 줄짜리 반복문으로 내적, 각 벡터의 노름을 동시에 계산
- 라이브러리 없이 순수 Java로 20줄 안에 구현 가능하다는 걸 보여주는 목적

문장 비교 실습 결과

- 기준: "이 커피메이커의 물탱크 용량이 궁금해요"
- 의미 유사 문장 → 유사도 0.62
- 의미 무관 문장 → 유사도 0.44
- 관찰 포인트: 표현이 달라도 의미가 비슷하면 유사도가 확실히 높다
- 표현은 완전히 다른데("물통에는 물이 얼마나 들어가나요?") 유사도가 높게 나온다는 게 핵심 증거

트러블슈팅 체크리스트

- Ollama 연결 실패 → ollama serve 실행 확인
- 모델 미다운로드 → ollama pull bge-m3 확인
- Netty DNS 관련 ERROR 로그는 무시해도 됨 (기능에 영향 없음)

도전 과제: 유사도 행렬 만들기

- 지금까지는 문장 2개씩만 비교했음 — 이번엔 5개 이상 문장으로 전체 조합을 비교해볼 것
- 조건: 그중 최소 2쌍은 표현은 다르지만 의미가 비슷한 문장으로 준비
- 힌트: 이중 for문으로 모든 문장 쌍의 코사인 유사도를 계산해서 $N \times N$ 행렬로 출력
- 힌트: 대각선(자기 자신과의 유사도)은 항상 1.0에 가까운지도 확인해볼 것
- 정답 코드는 제공하지 않음 — 예상과 다르게 나오는 쌍이 있다면 왜 그런지 직접 추론해볼 것
- 시간: 15~20분

M1.4 문서 로딩과 청킹 전략

왜 문서를 쪼개는가

- 컨텍스트 길이 제한 — LLM 프롬프트에는 최대 토큰 수가 있음 (예: llama2는 4K, GPT-4는 8K~128K)
- 검색 정밀도 문제 — 50페이지 문서를 벡터 하나로 뭉뚱그리면 "이 문서는 대충 이런 내용"이라는 평균적 의미만 담기고, "제3조 환불 정책은?"이라는 세부 질문에는 정확히 답할 수 없음
- 비유: 책 한 권을 통째로 색인 카드 한 장에 요약하는 것과, 페이지마다 색인 카드를 만드는 것 — 후자라야 "3장에 나온 그 얘기"를 바로 찾을 수 있음
- 실제로 이 차이를 눈으로 확인하는 건 잠시 후 Lab1.2에서 청킹 전후 검색 결과를 비교하며 직접 해봄

Spring AI DocumentReader 종류

TextReader

일반 텍스트 파일용

PagePdfDocumentReader

PDF 파일용, 페이지 단위로 Document 생성

TikaDocumentReader

Word, HTML 등 다양한 포맷을 아파치 티카로 파싱

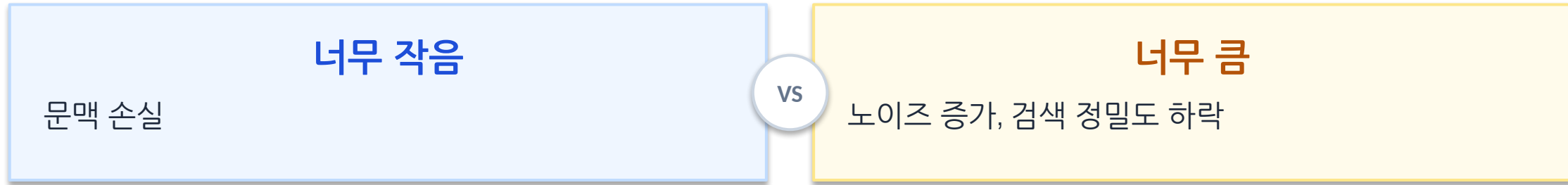
Document 객체 구조

- content: 본문 텍스트 (예: "환불은 14일 이내 가능합니다")
- metadata: 출처/페이지/작성일 등 부가정보를 Map 형태로 자유롭게 추가 (예: {"source": "manual.pdf", "page": 3, "chapter": "환불정책"})
- metadata의 용도: 나중에 검색 결과 옆에 "[manual.pdf, p.3]" 같은 출처 표기(citation)를 자동으로 붙일 때, 그리고 "이 섹션 문서만" 같은 필터 검색을 할 때 핵심 역할
 - 청킹 후에도 원본 Document의 metadata는 자식 청크들에게 자동 상속됨 — 때문에 출처 추적이 끊기지 않음

TextSplitter / TokenTextSplitter

- `TokenTextSplitter.builder().withChunkSize(N).build()`
 - 토큰 수 기준으로 자르기 때문에 실제 LLM이 보는 단위와 일치함
- `apply(documents)`를 호출하면 Document 리스트가 더 잘게 쪼개진 Document 리스트로 반환됨

Chunk size 트레이드오프



- 예: 한 문장씩 자르면 앞뒤 맥락이 사라지고, 문서 전체를 한 청크로 하면 아까 본 "평균화" 문제가 재발

Chunk overlap

- 문제: 청크 경계에서 문장이 절단되면 의미가 손상됨 (예: "환불은 14일 이내 가능하지만, 제품 결함은 30일" 처럼 경계에 끊김)
- 해결책: 인접한 청크끼리 마지막 N개 토큰을 중복 포함시키는 기법 (예: overlap=50 설정하면 청크A의 마지막 50토큰이 청크B 시작 50토큰과 겹침)
- 효과: 경계에 걸친 문장이 최소한 한쪽 청크에는 온전히 담김 → 검색 정확도 향상
- 트레이드오프: 저장 공간과 임베딩 비용이 조금 늘어나지만(overlap만큼 중복 저장), "경계에서 의미가 잘리는 문제"를 막는 대가로는 저렴한 편이라 실무에서 자주 사용

메타데이터 보존의 중요성

- 앞서 본 metadata 자동 상속이 실제로 왜 중요한가: 출처 표기(citation)와 필터 검색이 이것 하나에 달려있음
- metadata 없이 청킹하면? → 검색은 되지만 "이 답변이 어느 문서, 몇 페이지에서 왔는지" 영영 알 수 없게 됨
- 실전 예: "환불 정책은 제3조에 있습니다"처럼 출처를 밝히는 답변은 metadata가 살아있어야만 가능

잘못된 청킹의 실패 사례

- 문장 중간에서 절단되어 의미가 왜곡되는 실제 예시
- 예: "환불은 14일 이내 가능하지만, 제품 결함의 경우 30일까지"에서 앞부분만 잘려 "환불은 14일 이내 가능"만 검색되면 결함 시 예외 조항을 놓침

Lab 1.2 — 인덱싱 파이프라인 구현

- 목표: PDF 문서를 로드 → 토크닝 → VectorStore 저장 → 검색까지 전체 파이프라인 완성
- 실습 문서: 가상 제품(MCM-200 커피메이커) 매뉴얼 — 조항 번호/오류코드 포함
- 이 문서는 Day2 하이브리드 검색 실습에서도 그대로 재사용됨

PDF 로드

- documentPath는 classpath:/docs/manual.pdf 형태로 지정
- get()을 호출하는 순간 실제 파일 I/O와 PDF 파싱이 일어남

```
PagePdfDocumentReader pdfReader =  
    new PagePdfDocumentReader(documentPath);  
List<Document> documents = pdfReader.get();
```

칭킹 (chunk size 조정)

- 실습 문서가 짧아서 chunk size를 300 정도로 작게 잡아야 여러 개로 쪼개짐
- apply()는 내부적으로 각 Document의 content를 토큰화한 뒤 크기에 맞춰 분할

```
TokenTextSplitter splitter = TokenTextSplitter.builder()  
    .withChunkSize(300)  
    .build();  
List<Document> chunks = splitter.apply(documents);
```

청크 결과 확인 (실행 결과)

- 실측: 문서 1페이지 → 4개 청크로 분할됨
- 각 청크가 제1조/제3조/제5조/제7조 단위로 자연스럽게 나뉨 (조항 경계와 우연히 잘 맞아떨어진 사례)

SimpleVectorStore 구성 + 저장

- SimpleVectorStore는 인메모리 구현체 — 애플리케이션 재시작하면 데이터가 사라짐
- add(chunks) 호출 시 각 청크마다 내부적으로 embed()가 자동 호출됨 (직접 embed 호출 불필요)

```
VectorStore vectorStore =  
    SimpleVectorStore.builder(embeddingModel).build();  
vectorStore.add(chunks);
```

검색 결과 확인 (실행 결과)

- 질문: "환불 정책이 뭐야?"
- 실측: 제3조(환불 정책) 청크가 정확히 1순위로 검색됨
- similaritySearch() 호출 한 줄로 4개 청크 중 정답 청크가 상위로 올라옴을 직접 확인

[옵션] chunk size 바꿔가며 비교 실험

- chunk size를 늘리거나 줄여보며 검색 결과가 어떻게 달라지는지 직접 실험
 - 예: chunk size를 50으로 줄이면 청크 수가 늘어나지만 문맥이 부족해질 수 있음, 1000으로 늘리면 청크가 하나로 합쳐질 수 있음

트러블슈팅 & 정리

- PDF 못 찾을 → classpath: 접두어와 실제 리소스 폴더 구조가 일치하는지 확인
- 검색 결과가 빈 리스트 → `vectorStore.add(chunks)` 호출을 빼먹은 경우가 대부분
- 오늘 완성한 것: PDF 로드 → 토크닝 → 저장 → 검색까지 인덱싱 파이프라인 전체
- 남은 것: 검색된 결과로 실제 답변을 생성하는 것 (다음 M1.5)

도전 과제: 최적 chunk size 자동 탐색

- 지금은 chunk size 300으로 고정 — 이번엔 50부터 1000까지 자동으로 바꿔가며 실험할 것
- 목표: 같은 질문("환불 정책이 뭐야?")에 대해 검색 1등 결과가 chunk size에 따라 어떻게 바뀌는지 관찰
- 힌트: for문으로 chunk size 후보들을 순회하며, 매번 새로운 TokenTextSplitter와 SimpleVectorStore를 만들어야 함 (재사용하면 이전 청크가 섞임)
- 힌트: 결과가 "안정적으로 정답을 찾는" chunk size 구간이 있는지, 너무 작거나 커서 실패하는 구간이 있는지 표로 정리
- 정답 코드 없음 — 실험 결과를 근거로 우리 문서에 맞는 chunk size를 스스로 정해볼 것
- 시간: 20분

M1.5 검색과 생성 연결

지금까지의 한계

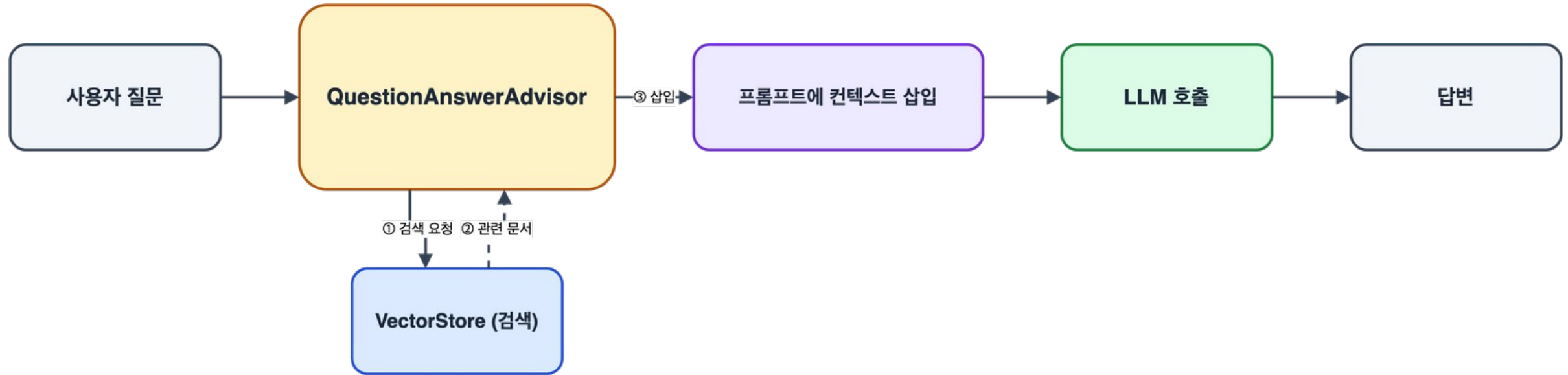
- 지금까지는 Retrieval(검색)만 완성 — 검색 결과로 "제3조(환불 정책) 청크"를 사용자에게 그냥 던짐
- 사용자가 원하는 건 "검색 결과를 정리해서 자연스럽게 답변해주는 것"인데, 우리는 아직 "원본 문장 조각"만 반환 (예: 사용자가 "환불은 몇 일?" → 우리가 반환: "환불은 14일 이내 가능하지만, 제품 결함의 경우 30일까지 가능합니다" 텍스트 그 자체 → 사용자가 원하는 건: LLM이 "14일이며, 결함 시 30일까지 가능합니다"로 요약해서 답변)
- 이것이 바로 "R"(검색)과 "AG"(증강+생성)의 차이 — 검색만 하면 도서관 사서, 검색+생성 연결까지 하면 비서

ChatClient와 Advisor 패턴

- 요청 전/후를 가로채서 처리를 끼워넣는 구조
- 서블릿 필터나 스프링 인터셉터와 비슷한 개념 — 요청이 실제 LLM에 도달하기 전에 가로채서 뭔가를 추가함

QuestionAnswerAdvisor 동작 원리

- 질문 → 자동으로 VectorStore 검색 → 검색 결과를 프롬프트에 삽입 → LLM 호출
- 개발자는 Advisor를 등록만 하면 이 4단계가 전부 자동으로 일어남



RetrievalAugmentationAdvisor vs QuestionAnswerAdvisor

- QuestionAnswerAdvisor: 빠르게 시작하는 기본형
- RetrievalAugmentationAdvisor: Query Transform/Rerank 등을 조립할 수 있는 확장형 (Day2에서 사용)
- 오늘은 QuestionAnswerAdvisor로 충분 — 확장형은 검색 품질을 튜닝해야 할 때 필요

프롬프트 템플릿 실제 모습

- 기본 QuestionAnswerAdvisor 템플릿을 그대로 화면에 띄워서 컨텍스트가 어디에 삽입되는지 보여줌
- 템플릿에는 {query}와 {question_answer_context} 두 개의 플레이스홀더가 있고, 이 자리에 실제 값이 채워짐

Advisor 체인 조합

- Advisor는 하나만이 아니라 여러 개를 동시에 적용 가능 (서블릿 필터 체인과 동일)
 - 자주 쓰이는 조합: SimpleLoggerAdvisor (요청/응답 로깅) + MessageChatMemoryAdvisor (이전 대화 기억) + QuestionAnswerAdvisor (RAG 검색)
 - 여러 Advisor를 등록하면 지정된 order대로 순차 실행 — 각 단계가 이전 Advisor의 결과를 받아서 가공 (예: 로깅 → 메모리 적용 → RAG 검색 → 최종 답변)
 - 실제 구성 예: `chatClient.advisors(loggerAdvisor, memoryAdvisor, qaAdvisor).prompt()...` 순서대로 세 Advisor가 파이프라인 형성

정리 + Lab1.3 예고

- ChatClient + Advisor 패턴 덕분에 RAG 파이프라인을 직접 손코딩할 필요 없이 QuestionAnswerAdvisor 하나만 등록하면 됨
 - 종합: VectorStore에 문서 저장(Lab1.2) → QuestionAnswerAdvisor에 연결 → ChatClient에 등록 = 완전한 RAG 챗봇
 - 다음: Lab1.3에서 이것 직접 조립해서 콘솔로 대화하는 챗봇을 완성

Lab 1.3 — 첫 RAG 챗봇 구현

- 목표: 콘솔에서 대화하는 RAG 챗봇 완성
- Lab1.2에서 만든 인덱싱 코드 + M1.5에서 배운 Advisor를 그대로 조립하는 실습

ChatClient + QuestionAnswerAdvisor 구성 + Q&A 루프

- defaultSystem(...)은 모든 요청에 공통으로 적용되는 시스템 프롬프트
 - 실측: llama3.2:3b가 가끔 영어/한자를 섞어 답하는 문제가 있어 이 설정으로 해결
- .advisors(qaAdvisor)를 호출한 요청만 검색이 적용됨 — 안 붙이면 그냥 순수 LLM 호출

```
ChatClient chatClient = ChatClient.builder(chatModel)
    .defaultSystem("항상 한국어로 답변하세요.")
    .build();
QuestionAnswerAdvisor qaAdvisor =
    QuestionAnswerAdvisor.builder(vectorStore).build();

// Scanner로 질문 받고 chatClient.prompt().advisors(qaAdvisor)...
```

성공 케이스 실행 결과

- 질문: "물탱크 용량이 얼마야?" → 답변: "물탱크 용량은 1.2리터입니다." (문서 근거)
- 질문: "E2 오류는 뭐야?" → 답변: 문서 제5조 내용대로 정확히 답함
- 두 질문 다 문서에 있는 단어를 거의 안 썼는데도 정확한 청크를 찾아 답변에 반영함

실패 케이스 관찰

- 시연 질문 예: "이 제품 방수 등급이 IP68 맞지?" (문서에 없는 내용을 사실처럼 질문)
- 관찰 포인트: 모델이 여전히 그럴듯하게 답을 지어낼 수 있음 → Day2/M2.6 "모른다고 답하기"로 연결
- 이건 RAG의 결함이 아니라 "아직 안 배운 것"의 예고편이라는 것을 명확히 짚어줄 것

도전 과제: 매번 다시 인덱싱하지 않게 만들기

- 지금 챗봇은 실행할 때마다 PDF를 다시 읽고 다시 임베딩함 — 문서가 커지면 매번 몇 초~몇 분씩 낭비
- 목표: 첫 실행에서 만든 벡터를 파일로 저장했다가, 다음 실행부터는 파일에서 불러오도록 수정
- 힌트: SimpleVectorStore는 save(File) / load(File) 메서드를 이미 제공함 (Lab1.1에서 다룬 적 있는 클래스)
- 힌트: 파일이 이미 존재하면 인덱싱을 건너뛰고 load()만, 없으면 인덱싱 후 save()하는 분기 로직 필요
- 정답 코드 없음 — 두 번째 실행부터 시작 속도가 실제로 빨라지는지 직접 확인할 것
- 시간: 15분

Day1 마무리

- 오늘 만든 것: 임베딩 이해 → 인덱싱 파이프라인 → 첫 RAG 챗봇
- 다음: Day2 — 검색 품질을 실전 수준으로 끌어올리기 (PGVector, 하이브리드 검색, rerank, query rewriting)
- 오늘 만든 코드는 전부 Day2에서 그대로 이어서 고도화됨 — 새로 시작하는 게 아님